

---

# Search Engine Project

## Stage 1: Building the Data Layer

Python implementation — technical report

---

|               |                                                                                                                                                                   |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Course        | Big Data · Grado en Ciencia e Ingeniería de Datos · Universidad de Las Palmas de Gran Canaria                                                                     |
| Academic year | 2026/2027                                                                                                                                                         |
| Group name    | BigData-Ulpgc                                                                                                                                                     |
| Members       | Daniel Perdomo Medina — Python implementation, author of this report<br>Daniel Rodríguez Alonso — Java implementation<br>Carlos Falcón Brito — C++ implementation |
| Repository    | <a href="https://github.com/BigData-Ulpgc/stage_1">https://github.com/BigData-Ulpgc/stage_1</a> , branch main, folder <code>python/stage_1/</code>                |
| Status        | Complete pipeline writing the three datalakes and the three indexes, offline mode with the sample dataset, 12 benchmark experiments, 39 tests passing             |
| Date          | 4 October 2026                                                                                                                                                    |

**Quick check for instructors.** From `python/stage_1/`, with Python 3.10+ and no network: `pip install -r ../requirements.txt pytest, pytest src/test`, then `python -m src.main.bigdata.main pipeline 15 --offline-source ../../sample_dataset/raw` and `python -m src.main.bigdata.main search whale island`, which finds books 76, 84 and 2701. Section 6.3 gives the details.

## Abstract

This report describes the Python implementation of Stage 1 of the group’s search engine over Project Gutenberg books. Stage 1 builds the *data layer*: a pipeline that downloads books, splits each one into header and body, stores them in a datalake, extracts their metadata into a SQLite datamart and builds an inverted index that answers Boolean AND queries. A control layer records the progress, so an interrupted run resumes without losing or repeating books.

The Python pipeline writes every book to the three datalake layouts (**book**, **range**, **time**) and to the three persistent inverted indexes (**monolithic** JSON, **hierarchical** folders, **mongo**) in the same run. It follows the group’s common contract (**shared/SPEC.md**): with the 200 books of the dataset its indexes are byte-for-byte identical to Java’s, and its random sequences reproduce Java’s exactly. A 15-book sample dataset lets the whole pipeline run offline. The 12 benchmark experiments were run on the 200 real books, on the same Linux machine as the Java ones. The **monolithic** index is the fastest to build (1.4 s for 200 books), to query (30  $\mu$ s per AND query) and to update (0.67 s per book), but it is the only one whose memory grows with the index (147 MB of Python objects for an 8.9 MB JSON). **hierarchical** spends 73 times its data size in file-system blocks, and the **time** datalake pays for its arrival-order layout with lookups 14 times slower than **book**. Next to Java, on the same machine, Python writes and builds at a similar speed but is 6–8 times slower wherever an interpreted loop runs per query or per term.

## Contents

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction and objectives</b>                         | <b>3</b> |
| 1.1      | Goal of Stage 1 . . . . .                                  | 3        |
| 1.2      | The group and the common contract . . . . .                | 3        |
| 1.3      | Objectives and scope of this report . . . . .              | 4        |
| <b>2</b> | <b>System architecture</b>                                 | <b>5</b> |
| 2.1      | The life of a book . . . . .                               | 5        |
| 2.2      | Layers and dependencies . . . . .                          | 6        |
| 2.3      | Design principles applied throughout . . . . .             | 6        |
| 2.4      | Project organisation . . . . .                             | 6        |
| 2.4.1    | Folder tree . . . . .                                      | 6        |
| 2.4.2    | Size of the code . . . . .                                 | 7        |
| 2.4.3    | Where each part of the state lives . . . . .               | 7        |
| <b>3</b> | <b>Design decisions</b>                                    | <b>7</b> |
| 3.1      | Structures selected for the final implementation . . . . . | 8        |
| 3.2      | Implementation decisions . . . . .                         | 8        |
| <b>4</b> | <b>Implementation: modules and classes</b>                 | <b>9</b> |
| 4.1      | Module models: the data that travels . . . . .             | 9        |
| 4.2      | Package <b>crawler</b> : fetching and splitting . . . . .  | 9        |
| 4.3      | Package <b>datalake</b> : storing the books . . . . .      | 10       |

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| 4.4      | Package <code>datamart.metadata</code> : metadata in SQLite . . . . . | 10        |
| 4.5      | Package <code>datamart.index</code> : the inverted index . . . . .    | 11        |
| 4.6      | Package <code>query</code> : the search . . . . .                     | 11        |
| 4.7      | Package <code>control</code> : progress and recovery . . . . .        | 12        |
| 4.8      | The command line: <code>main.py</code> . . . . .                      | 12        |
| 4.9      | Package <code>benchmark</code> . . . . .                              | 12        |
| <b>5</b> | <b>Testing strategy</b>                                               | <b>13</b> |
| 5.1      | Checks beyond the unit tests . . . . .                                | 13        |
| <b>6</b> | <b>Setup and execution</b>                                            | <b>14</b> |
| 6.1      | Requirements . . . . .                                                | 14        |
| 6.2      | Setup and tests . . . . .                                             | 14        |
| 6.3      | Quick test without network: the sample dataset . . . . .              | 14        |
| 6.4      | Commands . . . . .                                                    | 15        |
| 6.5      | Running the benchmarks . . . . .                                      | 15        |
| <b>7</b> | <b>Benchmarks and results</b>                                         | <b>15</b> |
| 7.1      | Methodology . . . . .                                                 | 15        |
| 7.2      | Data: real and synthetic . . . . .                                    | 16        |
| 7.3      | Datalake benchmark . . . . .                                          | 16        |
| 7.4      | Metadata benchmark . . . . .                                          | 17        |
| 7.5      | Inverted-index benchmark . . . . .                                    | 18        |
| 7.6      | Comparison with Java and C++ . . . . .                                | 20        |
| 7.7      | Summary: which structure to choose . . . . .                          | 23        |
| <b>8</b> | <b>Conclusions and future improvements</b>                            | <b>23</b> |
| 8.1      | Conclusions . . . . .                                                 | 23        |
| 8.2      | Limitations and future improvements . . . . .                         | 23        |

# 1. Introduction and objectives

## 1.1 Goal of Stage 1

The course project builds, stage by stage, a search engine over the books of Project Gutenberg. Stage 1 leaves ranking and user interfaces for later and concentrates on the data underneath it. The books have to be downloaded and kept, their metadata extracted, and their words indexed. This has to be done *correctly*, it has to *survive interruptions* without losing or repeating work, and it has to be *measured*, so that different ways of storing the same data can be compared with numbers.

| Layer        | What it does                                                                                                                                   | Python package          |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------|
| Ingestion    | Fetches the raw text (Gutenberg or a local folder) and splits it at the START/END markers into header and body; the licence footer is dropped. | <code>crawler</code>    |
| Datalake     | Keeps header and body as text files, in three layouts at once: <code>book</code> , <code>range</code> and <code>time</code> .                  | <code>datalake</code>   |
| Datamarts    | Data ready to query: the metadata in SQLite, and the inverted index (term → books) in three variants.                                          | <code>datamart.*</code> |
| Query        | AND search: the books that contain every term of the query.                                                                                    | <code>query</code>      |
| Control      | Two lists of ids, downloaded and indexed, that let a stopped run continue where it was.                                                        | <code>control</code>    |
| Command line | The pipeline loop and the <code>pipeline</code> , <code>search</code> and <code>status</code> commands.                                        | <code>main.py</code>    |
| Benchmarks   | The 12 experiments of the SPEC, their checks and the CSV results.                                                                              | <code>benchmark</code>  |

Table 1: The layers of the system and where each lives in the Python code.

## 1.2 The group and the common contract

The same data layer is written three times, in Java, Python (this report) and C++. Results from three languages can only be compared if the three programs read the same books, process them in the same way and write the same outputs, so the group wrote a common contract, `shared/SPEC.md`. Everything in it is fixed for the three implementations; everything else is up to each one.

| SPEC section     | What it fixes                                                                                                                                                                                                                               |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. Inputs        | <code>book_ids.txt</code> (the 200 ids, in download order), <code>queries.txt</code> (the query workload) and <code>stopwords.txt</code> ; blank lines and lines starting with <code>#</code> are skipped.                                  |
| 2. Split         | The START and END markers, with THE or THIS; without both, the book is discarded. The header is what comes before START, the body runs from the end of the START line to END, both trimmed, and <code>\r\n</code> becomes <code>\n</code> . |
| 3. Datalake      | The paths of the three layouts: <code>YYYYMMDD/HH/&lt;ID&gt;.body.txt</code> , <code>&lt;ID&gt;/body.txt</code> and <code>&lt;START&gt;--&lt;END&gt;/&lt;ID&gt;.body.txt</code> , with <code>START = ⌊ID/1000⌋ · 1000</code> in 5 digits.   |
| 4. Metadata      | Four regexes (title, author, release date, language): first match, first line, trimmed, NULL if absent; the <code>books</code> table with indexes on <code>author</code> and <code>title</code> .                                           |
| 5. Tokenizer     | Character by character: A–Z is lowered, a–z and 0–9 build tokens, any other character separates; tokens shorter than 2 and stopwords are dropped; a book contributes its <i>set</i> of terms.                                               |
| 6. Index         | <code>inverted_index.json</code> , <code>inverted_index/&lt;LETTER&gt;/&lt;term&gt;.txt</code> (one id per line) and the Mongo collection <code>inverted_index</code> ; postings sorted and without duplicates.                             |
| 7. Queries       | Same tokenizer; AND = intersection; ids in ascending order.                                                                                                                                                                                 |
| 8. Control       | <code>downloaded_books.txt</code> and <code>indexed_books.txt</code> ; an id is appended only after its step was written.                                                                                                                   |
| 9–10. Benchmarks | The CSV format, 2 warm-ups + 5 runs, no network inside a measurement, the N books with the lowest ids, the reference term counts, and the generated metadata rows.                                                                          |

Table 2: The common contract, `shared/SPEC.md`, in short.

### 1.3 Objectives and scope of this report

The assignment asks for a datalake of raw books, datamarts with the metadata and the inverted index, a control layer that never loses or duplicates work, and benchmarks of several datalake and index structures, implemented in at least three languages over the same data.

This report covers the **Python** implementation. It explains its architecture, the decisions behind it and each of its modules, how to install, test and run it, and what its benchmarks show, on their own and next to Java and C++. The other two implementations are described in their own reports in the repository.

The Python implementation delivers:

- The complete pipeline (crawler, datalake, metadata, inverted index, AND search and control layer) behind a command line with `pipeline`, `search` and `status`.
- The three datalake layouts and the three persistent indexes of the SPEC, all written by the same pipeline run, plus an in-memory index used as the reference in the benchmark checks.
- An offline source (`--offline-source`) that runs the same pipeline on the raw files of `sample_dataset/`, without network.
- The 12 benchmark experiments of the SPEC in its CSV format, with internal checks that stop a run that measures something wrong, and `verify_results`, which checks the CSVs against the exact values of SPEC section 10.
- 39 `pytest` tests.

## 2. System architecture

Figure 1 follows one book through `main.py`. Unlike Java and C++, which choose one datalake and one index in a configuration file, the Python pipeline writes the book to every structure in the same pass.

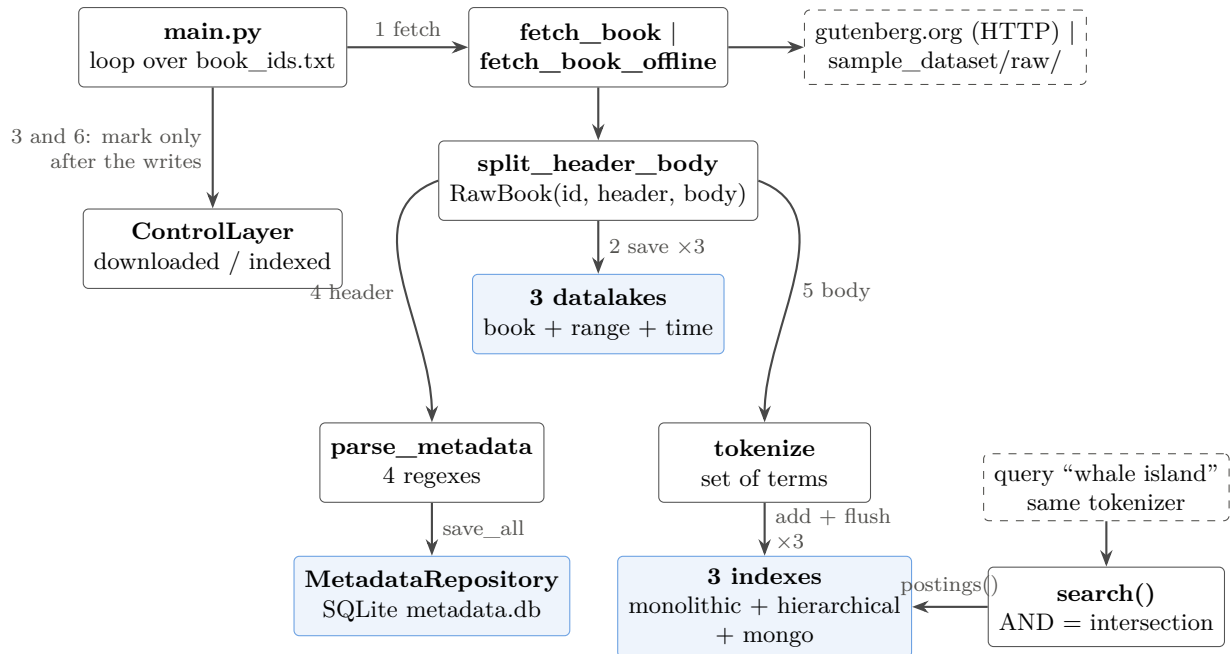


Figure 1: Main components and data flow of one book. The numbers follow the order of `main.py`; Mongo is skipped, with a warning, when no server answers.

### 2.1 The life of a book

1. `main.py` reads `shared/book_ids.txt` and goes through the ids in order. Book 1342 is not in `indexed_books.txt`, so it is processed; an id that is already there is skipped and does not count as a step.
2. `fetch_book` downloads `pg1342.txt` with `requests` (or `fetch_book_offline` reads it from a folder), turns `\r\n` into `\n` and calls `split_header_body`, which finds the markers and returns header and body, both trimmed. A book without both markers is discarded.
3. The `RawBook` is saved in the three datalakes. Each file is written to a `.tmp` and moved into place, so a book is either complete or absent. Only then is the id appended to `downloaded_books.txt`, by `mark_as_downloaded(1342)`.
4. `parse_metadata` applies the four regexes of the SPEC to the header, and `MetadataRepository.save_all` stores the row in SQLite.
5. `tokenize` turns the body into its set of terms, which is added to the monolithic, the hierarchical and (with a server) the Mongo index. The three are **flushed** to disk right away.
6. Only after the three flushes, `mark_as_indexed(1342)` appends the id to `indexed_books.txt`.
7. A query “whale island” goes through `search`: it is tokenized like a body, the posting lists of `whale` and `island` are read from the index, and they are intersected, the shortest first.

## 2.2 Layers and dependencies

Every module only imports modules of the same or a lower layer. The storage modules know nothing about the pipeline, and the pipeline only uses them through their common methods (`save/locate/list_book_ids` for datalakes, `add_document/postings/flush` for indexes).

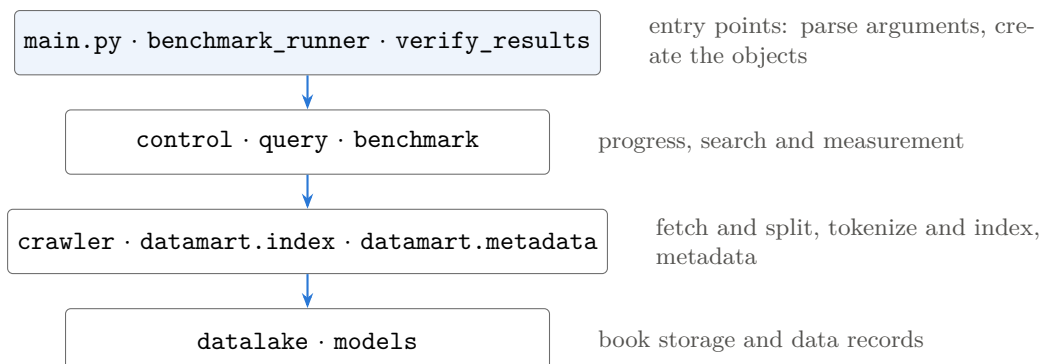


Figure 2: Layers. Each one only uses the layers below it.

## 2.3 Design principles applied throughout

| Principle               | Where                                                                                                 | Why                                                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Abstract classes        | base <b>Datalake</b> and <b>InvertedIndex</b> ( <code>abc.ABC</code> )                                | Every structure offers the same methods, so the pipeline, the search and the benchmarks treat them alike.               |
| Dependencies passed in  | Paths, the clock of <b>TimeBasedDatalake</b> , the Mongo database name                                | Tests and benchmarks use temporary folders, a simulated clock and <b>search_engine_bench</b> without touching the code. |
| One way to write a file | <b>Datalake._write_file</b> : subclasses decide <i>where</i> , the base decides <i>how</i> .          | The delicate part (the atomic write) exists only once for the three layouts.                                            |
| Atomic writes           | Datalake files, <b>inverted_index.json</b> , term files, CSV results                                  | A crash never leaves a half-written file under its final name.                                                          |
| Idempotence             | <b>INSERT OR REPLACE</b> , sets in <b>add_document</b> , <b>\$addToSet</b> , marks that check first   | Processing a book twice after a failure adds nothing.                                                                   |
| Mark after doing        | <b>mark_as_downloaded</b> and <b>mark_as_indexed</b> come last                                        | The control layer never claims something that was not written.                                                          |
| Immutable records       | <b>RawBook</b> , <b>BookLocation</b> , <b>BookMetadata</b> : frozen dataclasses with <b>__slots__</b> | Plain data that cannot be changed by accident, and small in memory.                                                     |

Table 3: Design principles and where they appear in the code.

## 2.4 Project organisation

### 2.4.1 Folder tree

```

stage_1/
|-- README.md           overview of the group project
|-- docker-compose.yml  MongoDB 8.2 server shared by the three implementations
|-- shared/             COMMON CONTRACT (Java, Python and C++)

```

```
| |-- SPEC.md, book_ids.txt, queries.txt, stopwords.txt
|-- sample_dataset/      first 15 books, raw (input) and split (expected output)
'-- python/
    |-- requirements.txt   requests, pymongo
    |-- stage_1/          <- the Python implementation (run everything from here)
        |-- README.md     user guide of the module
        |-- docs/         this report; report/ holds its LaTeX source and scripts
        |-- src/main/bigdata/
            |-- main.py    command line: pipeline | search | status
            |-- models.py  RawBook, BookLocation, BookMetadata (dataclasses)
            |-- crawler/   client.py (HTTP), splitter.py (markers, offline read)
            |-- datalake/  base.py, book_based.py, range_based.py, time_based.py
            |-- datamart/metadata/ parser.py, repository.py (SQLite)
            |-- datamart/index/ tokenizer.py, base.py, monolithic.py, hierarchical.py,
            |              mongo.py, in_memory.py
            |-- query/     search_service.py (AND)
            |-- control/   state_manager.py (ControlLayer)
            |-- benchmark/ benchmark_runner.py, datalake_/index_/metadata_benchmark.py,
            |              runner.py, books.py, csv_results.py, verify_results.py
        |-- src/test/     39 tests (pytest), one package per source package
        |-- data/         (generated, ignored by git) datalake, datamarts, control
        '-- benchmarks/   results/{real,synthetic}/*.csv (committed), data/ (ignored)
```

## 2.4.2 Size of the code

| Item                  | Amount                                                                                      |
|-----------------------|---------------------------------------------------------------------------------------------|
| Production modules    | 26 (about 2,900 lines with comments and docstrings)                                         |
| Test modules          | 14 (about 630 lines), 39 tests                                                              |
| External dependencies | requests 2.32, pymongo 4.18; pytest for the tests; matplotlib only for this report's charts |

## 2.4.3 Where each part of the state lives

| State              | File or database                                             | Written by                    |
|--------------------|--------------------------------------------------------------|-------------------------------|
| Book text          | data/datalake/{book,range,time}/...                          | Datalake.save                 |
| Metadata           | data/datamarts/metadata.db (table books)                     | MetadataRepository.save_all   |
| Monolithic index   | data/datamarts/ inverted_index.json                          | MonolithicJsonIndex.flush     |
| Hierarchical index | data/datamarts/inverted_index/ <L>/<term>.txt                | HierarchicalFolderIndex.flush |
| Mongo index        | database search_engine, collection inverted_index            | MongoInvertedIndex.flush      |
| Progress           | data/control/downloaded_books.txt, indexed_books.txt         | ControlLayer.mark_as_*        |
| Benchmark results  | benchmarks/results/{real,synthetic}/ python_<experiment>.csv | csv_results.write_csv         |

## 3. Design decisions

### 3.1 Structures selected for the final implementation

The Python pipeline keeps every structure up to date, so choosing one does not change what is stored; it decides which structure a search engine built on this layer should rely on.

**Datalake. book.** Its lookup is the fastest ( $27.8\mu\text{s}$  per book, against 30.0 for **range** and 377.9 for **time**), because the path is computed from the id; its write throughput (1,398 books/s) is within 5% of the fastest layout; it detects new books and recovers from an interrupted write as well as the others (20 recovered, 0 lost, 0 duplicated); and its storage overhead (1.5% of allocated blocks) is negligible. It is also the layout the benchmarks read (SPEC 10.1), and the one Java and C++ chose.

**Inverted index. monolithic,** at the scale of this stage. With 200 books it is the fastest to build (1.4 s, against 15.2 for **mongo** and 20.1 for **hierarchical**), to query ( $29.6\mu\text{s}$  against 171 and 988) and to update (674 ms per book against 1,723 and 3,230), and the smallest on disk (8.9 MB). Its weak point is that it lives in memory: 147 MB of Python objects after reopening, growing with the index, and every flush rewrites the whole file, so its update cost grows linearly with  $N$ . For a much larger collection, **mongo** is the alternative: it keeps almost nothing in the client and answers several processes at once.

### 3.2 Implementation decisions

**The pipeline writes the three structures at once.**

Every book goes to the three datalakes and the three indexes in the same pass. One **pipeline** 200 run fills everything the benchmarks and the comparison need, and the structures can never drift apart, because they are built from the same book in the same step. The price is that each book is written six times; for 200 books this is a matter of minutes, and the download dominates anyway.

**Standard library, plus requests and pymongo.**

SQLite comes with Python (**sqlite3**), as do JSON, paths, **tracemalloc** and timing. Keeping the dependencies to two well-known packages makes the module easy to install on any machine, which matters because the official benchmarks were run on a different computer than the one used to write the code.

**A tokenizer that walks the text character by character.**

**tokenize** is a loop that compares each character with the ranges A–Z, a–z and 0–9, exactly as SPEC section 5 describes. A regex such as `\w+` or `str.lower()` would follow Unicode rules (accented letters, the underscore) and give different terms from Java and C++. With the loop, the index of the 200 books has exactly the 129,356 terms and 1,581,064 postings of the reference.

**Atomic writes with `.tmp` and `os.replace`.**

Datalake files, the monolithic JSON and each hierarchical term file are first written to `<name>.tmp` and then moved over the final name. **os.replace** is atomic within one file system on Linux and Windows, so a file under its final name is always complete, and a `.tmp` never counts as a saved book. The datalake recovery benchmark relies on it: it simulates a crash by leaving bodies as `.tmp`.

**Save each book before marking it (SPEC section 8).**

The three indexes are flushed after every book, and only then is the id appended to `indexed_books.txt`. Flushing the monolithic JSON after every book costs time, since the whole file is rewritten, but it is what makes the control layer truthful: if the run stops, every book marked as indexed is really on disk, and the worst case is indexing one book again, which adds nothing.

**A control layer that trims a half-written line.**

Ids are appended one per line. If the process dies in the middle of an append, the file can end in a fragment such as 13. `ControlLayer` only counts lines that end in `\n` and truncates such a fragment when it starts, as Java's `ControlFiles` does, so the next append cannot glue it to another id ( $13 + 84 = 1384$ ).

**JavaRandom and java\_shuffle.**

Python's `random` does not produce Java's sequence for the same seed. Where the comparison between languages depends on random choices, the lookup order of `datalake_lookup` and the rows picked by `metadata_query` (both with seed 42), Python uses a port of `java.util.Random` (the same 48-bit linear congruential generator and `nextInt` rejection loop) and of `Collections.shuffle`. They were checked against the JDK and give exactly its output.

**\$addToSet in MongoDB.**

A flush sends one upsert per pending term, all in one unordered `bulk_write`, with `$addToSet` and `$each`. Adding the same book twice leaves the posting list unchanged, which keeps the index correct when a book is processed again after an interruption. Because Mongo does not keep arrays sorted, `postings` sorts what it reads.

**Check every index against an in-memory one.**

After measuring, each index benchmark calls `verify()`, a port of Java's `IndexBenchmark.verify`: the index must give the same posting lists as an in-memory index built from the same books, for the query terms and the first 20 terms of the first and last book, and the same answer to every query. A benchmark that measured a wrong index stops with an error instead of writing numbers.

## 4. Implementation: modules and classes

For each module: what it is, what it is responsible for, its important functions, and the reason for its design. The order follows the path of the data. All paths are relative to `src/main/bigdata/`.

### 4.1 Module models: the data that travels

**RawBook** frozen dataclass

A downloaded book: `id`, `header` and `body`. It is what the splitter returns and what the datalakes save.

**BookLocation** frozen dataclass

Where a book is stored: its `id` and the paths of its header and body files.

**BookMetadata** frozen dataclass

A row of the `books` table: `id`, `title`, `author`, `language`, `release date` and the two paths.

*Design rationale.* `frozen=True` makes the records impossible to change by accident once created, and `slots=True` keeps them small when the benchmarks hold hundreds of thousands of them.

### 4.2 Package crawler: fetching and splitting

**client.download\_raw\_text** function

Downloads `https://www.gutenberg.org/cache/epub/<ID>/pg<ID>.txt` with `requests` (30 s timeout), decodes it as UTF-8 and turns `\r\n` into `\n`. A non-200 answer raises `DownloadError`.

**splitter.split\_header\_body** function

Finds the first line that contains the START marker and the first END marker after it, with case-sensitive regexes that accept THE and THIS. The header is everything before the START line and the body everything between the two marker lines, both trimmed. A missing marker raises `MarkerNotFoundError`.

**splitter.fetch\_book, fetch\_book\_offline** functions

Download (or read `<dir>/pg<ID>.txt`) and split, returning `None` instead of raising, so the pipeline only has to check one value. The offline variant reads the same raw bytes Gutenberg serves, so everything after it is identical to an online run.

*Design rationale.* The split is a pure function of the text: it can be tested without network, and the online and offline paths share it.

**4.3 Package datalake: storing the books****Datalake** abstract base class, `base.py`

The common interface: `name`, `save(book)`, `locate(id)` and `list_book_ids()`. It also holds the only file writer, `_write_file`: create the folder, write `<name>.tmp`, `os.replace` it over `<name>`, and delete the `.tmp` if anything fails. A book counts as saved only when its header and body both exist as regular files.

**BookBasedDatalake** layout book

`<ID>/header.txt` and `<ID>/body.txt`. `locate` computes the path from the id; listing keeps only canonical numeric folder names.

**RangeBasedDatalake** layout range

`<START>-<END>/<ID>.{header,body}.txt`, with `START = (id // 1000) * 1000` and five digits. Listing ignores files whose id does not belong to the range of their folder.

**TimeBasedDatalake** layout time

`YYYYMMDD/HH/<ID>.{header,body}.txt`, with the time given by a clock passed to the constructor (`datetime.now` by default). The path cannot be computed from the id, so `locate` searches the hour folders from the newest, and `list_book_ids` removes repeated ids.

*Design rationale.* Passing the clock in lets the benchmark use a simulated one (10 books per hour from 1 January 2026, UTC, as Java's `SimulatedClock`), so every repetition creates exactly the same folders.

**4.4 Package datamart.metadata: metadata in SQLite****parser.parse\_metadata** function

Applies the four multi-line regexes of SPEC section 4 to the header and returns a dictionary with `title`, `author`, `release_date` and `language`. Each value is the first match, first line only, trimmed; a missing or empty value is `None`, which SQLite stores as `NULL`. The release date drops whatever follows a bracket, such as `[eBook #1342]`.

**repository.MetadataRepository** class

Opens (or creates) `metadata.db` with the SPEC schema, optionally without the `author` and `title` indexes (the `sqlite_no_index` variant of the benchmark). `save_all(rows)` inserts a batch

with `executemany` and `INSERT OR REPLACE`, in one transaction; `find_by_id`, `find_by_author` and `find_by_title` return `BookMetadata` records, the last two ordered by `book_id`.

*Design rationale.* `INSERT OR REPLACE` makes saving a book twice harmless, and one commit per batch is what makes bulk inserts fast in SQLite.

## 4.5 Package `datamart.index`: the inverted index

**`tokenizer.tokenize`** function

The loop of SPEC section 5 (Section 3). It returns a `set`, since each book contributes each term once. The stopwords are read once, when the module is imported, from `shared/stopwords.txt`.

**`InvertedIndex`** abstract base class, `base.py`

Its methods are `add_document(id, terms)`, `postings(term)` (sorted ids), `flush()`, `clear()`, `disk_usage_bytes()` and `close()`. `add_document` only touches memory; nothing is persistent until `flush`.

**`InMemoryInvertedIndex`** class, structure `memory`

A dictionary of sets. It is never persisted: it is the reference against which `verify()` checks the other three.

**`MonolithicJsonIndex`** class, structure `monolithic`

Loads `inverted_index.json` when created and keeps the whole index in memory. `flush` serialises it with sorted keys, sorted ids and no spaces (`separators=(',', ':')`), writes it to `inverted_index.json.tmp` and moves it into place. The bytes are identical to Java's file.

**`HierarchicalFolderIndex`** class, structure `hierarchical`

One file per term in `<root>/<FIRST LETTER>/<term>.txt`, one id per line; terms that start with a digit go to the folder of that digit. `add_document` accumulates pending ids; `flush` reads each changed term's file, merges, and rewrites it (`.tmp` + move) only if some id was new. `postings` reads a single file and adds what is pending.

**`MongoInvertedIndex`** class, structure `mongo`

Collection `inverted_index` with a unique index on `term`, created in the constructor, which also checks that the server answers within 3 s. `flush` sends all pending terms in one unordered `bulk_write` of upserts with `$addToSet`. `disk_usage_bytes` asks for an `fsync` and returns `storageSize + totalIndexSize` from `collStats`.

*Design rationale.* The three persistent indexes keep pending work in memory and only write in `flush`, so the caller decides when the disk is touched: once per book in the pipeline, once per build in the benchmark.

## 4.6 Package `query`: the search

**`search_service.search`** function

Tokenizes the query, returns an empty list if no term is left, asks the index for every posting list (stopping at the first empty one), sorts them by length and intersects them with two pointers, the shortest first, so every step works on the smallest possible list. The result is sorted, as SPEC section 7 requires, and works with any `InvertedIndex`.

## 4.7 Package control: progress and recovery

**state\_manager.ControlLayer** class

Owns `downloaded_books.txt` and `indexed_books.txt`. `is_downloaded` and `is_indexed` read the complete lines of the file; `mark_as_downloaded` and `mark_as_indexed` append an id only if it is not there yet. When created, it truncates a last line left without `\n`. Valid ids follow the datalake's rule (0 or no leading zeros, at most 9 digits), and a Windows `\r` is ignored.

*Design rationale.* The files are the only memory of the progress between runs, so they are read from disk every time instead of being cached, and they are append-only.

## 4.8 The command line: `main.py`

`python -m src.main.bigdata.main` with three sub-commands (`argparse`): `pipeline [steps] [--offline-source DIR]`, `search WORDS...` and `status`. The pipeline loop is described in Section 2.1. A book that cannot be fetched or split counts as an error and is retried in the next run; one already indexed counts as skipped. MongoDB is optional: if the connection fails, the run continues with the other two indexes and says so. `search` reads the monolithic index.

Known limitation: a book that was downloaded but not indexed when a run stopped is downloaded again on the next run, instead of being read from the datalake as Java does. The result is the same, but in the `time` layout the book can then exist in two hour folders; `locate` returns the newest and `list_book_ids` reports the id once.

## 4.9 Package benchmark

**runner** module

`measure(experiment, structure, N, setup, task)` runs `setup` (not timed) and `task` (timed with `time.perf_counter_ns`) 2 + 5 times and returns one `BenchmarkRow` per measured run. `BenchmarkRow` formats the CSV line with at most 3 decimals. `JavaRandom` and `java_shuffle` also live here.

**csv\_results** module

Writes a whole CSV with LF line endings to `<file>.tmp` and moves it into place.

**books** module

`from_datalake` reads every book of a book datalake in ascending id order (SPEC 10.1); `synthetic` and `synthetic_zipf` generate books from a seed.

**datalake\_benchmark, index\_benchmark, metadata\_benchmark** modules

The 12 experiments (Section 7) and their checks.

**benchmark\_runner** entry point

With a datalake path, the real mode (10 CSVs in `results/real/`); without it, the synthetic mode (12 CSVs in `results/synthetic/`).

**verify\_results** entry point

Checks the CSVs against the exact values of SPEC section 10 (Section 5.1).

## 5. Testing strategy

`pytest src/test` runs 39 tests in 14 modules, which mirror the packages of the code. They need no network and no database server, and write only to temporary folders.

| Module                                         | What it checks                                                                                                                                                                                                                                         | Tests |
|------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| <code>crawler/test_splitter</code>             | The header is what precedes <code>START</code> , the body what lies between the markers, neither contains a marker, and the result is a (header, body) pair.                                                                                           | 5     |
| <code>control/test_state_manager</code>        | Marks written once; a half-written last line ignored and truncated, so the next id is not glued to it; a file without any newline; <code>\r\n</code> , garbage and leading zeros.                                                                      | 5     |
| <code>datalake/test_range_based</code>         | The range folder of ids 0, 999, 1530 and 2000.                                                                                                                                                                                                         | 4     |
| <code>datamart/test_tokenizer</code>           | Lowercasing, accented characters as separators, stopwords and the minimum length.                                                                                                                                                                      | 4     |
| <code>datamart/test_metadata</code>            | Title, author and language extracted; a missing release date is <code>None</code> ; the four keys always present.                                                                                                                                      | 3     |
| <code>datamart/test_hierarchical_index</code>  | A flush sorts out-of-order ids and removes duplicates.                                                                                                                                                                                                 | 2     |
| <code>datalake/test_book_and_time_based</code> | The book paths and content, <code>locate</code> and listing, a body left as <code>.tmp</code> not counted; the <code>time</code> layout with the simulated clock (10 books per hour) and a repeated book located in its newest folder and listed once. | 5     |
| <code>benchmark/test_java_random</code>        | <code>JavaRandom.next_int</code> and <code>java_shuffle</code> against values produced by the JDK with seed 42.                                                                                                                                        | 3     |
| <code>benchmark/test_index_verify</code>       | <code>verify()</code> accepts a correct index and stops on an index that loses a posting or a book.                                                                                                                                                    | 3     |
| Other five modules                             | Import checks of the runner, the metadata repository, the Mongo, monolithic and search modules.                                                                                                                                                        | 5     |

Table 4: The `pytest` suite.

**MongoDB.** The unit suite has no test that needs a server: `test_mongo_index` only checks that the module imports. The Mongo index is exercised against a real server by the benchmarks, where every experiment checks it with `verify()`, and by the pipeline checks of Section 5.1.

### 5.1 Checks beyond the unit tests

Most of the evidence that the Python module is correct comes from checks that run on the real data.

- **`verify()` in the five index experiments.** After `index_build` and `index_update`, on the reopened index in `index_query` and `index_memory`, and before writing the rows of `index_disk`, each backend is compared with an in-memory index built from the same books. A fresh index is also checked to be empty (no files, no postings for the query terms) every time an experiment starts from an empty index.
- **Datalake checks.** `datalake_lookup` requires every id to be found; `datalake_incremental` requires the detected ids to be exactly the new ones; `datalake_recovery` requires `recovered` = `damaged`, `lost` = 0 and `duplicates` = 0, computed on the final state of the folder. The metadata experiments check the number of inserted rows and that the 1,000 queries of each type return at least 1,000 rows.
- **`verify_results`.** Checks the committed CSVs: the exact storage, recovery, incremental

and index counts, and that every experiment has its 5 runs. It exits with code 1 if anything fails.

- **Parity with Java, checked on the output of pipeline 200.** The monolithic index has 8,893,537 bytes, 129,356 terms and 1,581,064 postings; the hierarchical index has 129,356 files and 7,251,967 bytes; every posting list is sorted, without duplicates and without `\r`, and each term file equals its JSON entry. These numbers are identical to Java's. The Mongo collection of `search_engine` has 129,356 documents and 1,581,064 postings. No file of the datalake contains a `\r`, and the three layouts hold 200 headers and 200 bodies each.
- **Java's random sequence.** `JavaRandom` and `java_shuffle` were run next to the JDK on a 200-id shuffle and 3,000 `nextInt` calls: the output is exactly the same.
- **End to end.** The real mode of the runner on the 15 books of `sample_dataset/book` finishes without errors, with the three index backends, and recovers 2 / 0 / 0 books.

## 6. Setup and execution

### 6.1 Requirements

- Python 3.10 or newer, and `pip`.
- `requests` and `pymongo` (`python/requirements.txt`); `pytest` for the tests.
- MongoDB, only for the `mongo` index: `docker compose up -d` at the repository root starts the group's MongoDB 8.2 on `127.0.0.1:27017`. Without it, the pipeline and the benchmarks skip that index with a warning.

### 6.2 Setup and tests

From `python/stage_1/`:

```
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r ../requirements.txt pytest
pytest src/test                   # 39 passed
```

### 6.3 Quick test without network: the sample dataset

`sample_dataset/raw/` holds the first 15 books of the dataset exactly as Gutenberg serves them. Starting from an empty `data/`:

```
python -m src.main.bigdata.main pipeline 15 --offline-source ../../sample_dataset/raw
python -m src.main.bigdata.main search whale island
[INFO] Found 3 books: [76, 84, 2701]
```

A later online pipeline finds those 15 books already indexed and downloads only the other 185.

## 6.4 Commands

| Command                                                            | (python -m src.main.bigdata.main ...) | What it does                                                                                                          |
|--------------------------------------------------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>pipeline [steps]</code>                                      |                                       | Processes up to <code>steps</code> books not yet indexed, in the order of <code>book_ids.txt</code> (all by default). |
| <code>pipeline [steps]</code><br><code>--offline-source DIR</code> |                                       | The same, reading <code>DIR/pg&lt;ID&gt;.txt</code> .                                                                 |
| <code>search WORDS...</code>                                       |                                       | AND search on the monolithic index; prints the ids.                                                                   |
| <code>status</code>                                                |                                       | Books in the dataset, downloaded, indexed, and pending (not indexed).                                                 |

A step is one book (download and indexing), so `pipeline 200` here does what `pipeline 400` does in Java and C++. With the whole dataset, `status` reports 200 downloaded, 200 indexed and 0 pending.

## 6.5 Running the benchmarks

The real mode reads a book datalake: `data/datalake/book` after `pipeline 200`, or Java's, which is byte-for-byte the same. Run the two modes one after the other, then check the CSVs:

```
python -m src.main.bigdata.benchmark.benchmark_runner data/datalake/book > benchmarks/
    bench_real.log 2>&1
python -m src.main.bigdata.benchmark.benchmark_runner > benchmarks/bench_synthetic.log 2>&1
python -m src.main.bigdata.benchmark.verify_results
```

Both logs must list the three backends, monolithic, hierarchical and mongo. The CSVs of each block (datalake, metadata, index) are written when the block ends, so the machine must not sleep during the run, which takes hours.

**Windows and Linux.** The commands are the same on both; on Windows the environment is activated with `venv\Scripts\activate`. The difference is speed: each repetition of the hierarchical index experiments creates and deletes about 129,000 small files, which is far slower on NTFS with an antivirus scanning every new file, and a folder synchronised by OneDrive makes it worse. The official results were therefore produced on Linux.

## 7. Benchmarks and results

### 7.1 Methodology

- **Warm-ups and measured runs:** 2 discarded repetitions and 5 measured ones, timed with `time.perf_counter_ns`. Tables give the **median** of the 5.
- **Setup outside the measurement:** deleting folders, filling the datalake or the index that an experiment starts from, opening connections and tokenizing the books happen in the untimed setup. What is timed: saving the books (`datalake_write`); every `locate` (`datalake_lookup`); listing the ids minus the known ones (`datalake_incremental`); listing and saving the missing books (`datalake_recovery`); the batches of 1,000 rows (`metadata_insert`); 1,000 queries of one type (`metadata_query`); `add_document` of the `N` books plus one flush (`index_build`); 100 rounds of the 10 queries of `queries.txt` (`index_query`); adding the last 10% of the books with one flush per book (`index_update`).

- **Same data and order as Java:** the books in ascending id order, N as a prefix, the 10% splits of Java, and `JavaRandom(42)` for the lookup order and the metadata queries. The `time` layout uses a simulated clock of 10 books per hour.
- **Verification:** every experiment checks its result (Section 5.1); a failed check stops the run before any CSV is written.
- **index\_memory:** measured with `tracemalloc`. It is the net difference of the live memory allocated through Python’s allocator, after `gc.collect()`, between before and after building the index (`heap_after_build`) and between before and after reopening it (`heap_after_open`). It does not include the memory of `mongod`, nor what C libraries allocate outside Python’s allocator, nor the books, which were loaded before tracing started. Negative values are kept, as in Java. It is not comparable in absolute value with the JVM heap that Java reports, only in how it grows with N.

**Environment.** Same Linux machine as the Java run (12 threads, 15 GB RAM, NVMe SSD), Python Python 3, MongoDB 8.2 in Docker (local). Absolute values depend on the machine; what matters are the ratios and how they change as N grows.

## 7.2 Data: real and synthetic

- **Real books (SPEC 10.1)** for the datalake and index experiments: the 200 books of `shared/book_ids.txt`, read from a book datalake (129.0 MB). The datalake uses all 200; the index uses the prefixes of 50, 100 and 200 books, with exactly 58,834, 78,820 and 129,356 terms.
- **Generated metadata (SPEC 10.2)** for the metadata experiments: 1,000, 10,000 and 100,000 rows built without random numbers (every author has 10 books and every title 2), identical in the three languages.
- **Synthetic books (SPEC 10.3):** the synthetic mode also runs the datalake and index experiments on 200 generated books (5,000 Zipf-distributed words from a vocabulary of 5,000). They come from Python’s `random`, so they are a Python-only reference and are not compared with the other languages.

## 7.3 Datalake benchmark

Data: the 200 real books, the same bytes in the three layouts.

| Experiment                        | What exactly is measured                                                                          | Metrics                              |
|-----------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------|
| <code>datalake_write</code>       | Saving the 200 books into an empty datalake.                                                      | elapsed (ms), throughput (books/s)   |
| <code>datalake_lookup</code>      | <code>locate()</code> of every id, in Java’s shuffled order (seed 42).                            | elapsed, per_lookup ( $\mu$ s)       |
| <code>datalake_incremental</code> | With 180 known and 20 new books: <code>list_book_ids()</code> minus the known ids.                | elapsed, detected                    |
| <code>datalake_recovery</code>    | 1 book in 10 has its body left as <code>.tmp</code> ; listing and saving the missing ones again.  | elapsed, recovered, lost, duplicates |
| <code>datalake_storage</code>     | After saving everything: files, folders, fullest folder, bytes and allocated bytes (4 KB blocks). | files, directories, ...              |

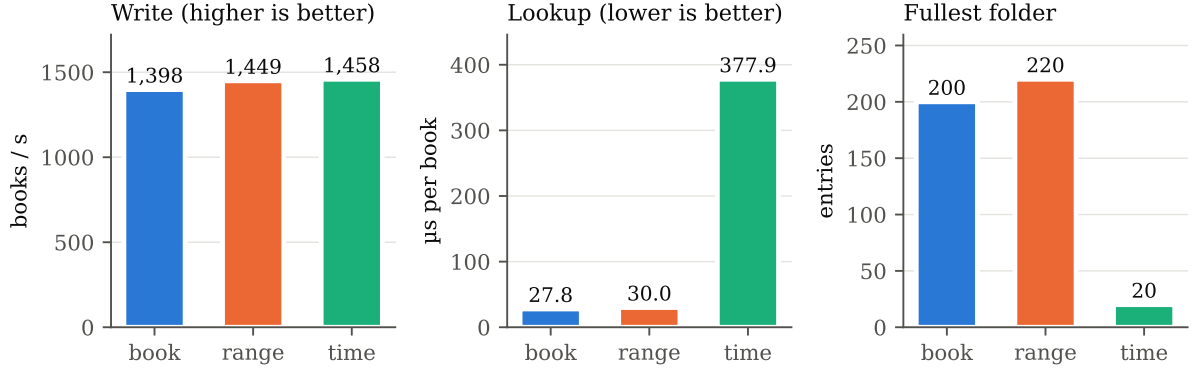


Figure 3: Datalake with the 200 real books (median of 5 runs).

| Metric                        | book       | range      | time       |
|-------------------------------|------------|------------|------------|
| Total write (ms)              | 143.1      | 138.0      | 137.1      |
| Write (books/s)               | 1,398      | 1,449      | 1,458      |
| Lookup per book ( $\mu$ s)    | 27.8       | 30.0       | 377.9      |
| Detect new books (ms)         | 6.28       | 6.87       | 6.34       |
| Recovery (ms)                 | 22.5       | 23.6       | 20.4       |
| Recovered / lost / duplicated | 20 / 0 / 0 | 20 / 0 / 0 | 20 / 0 / 0 |
| Files                         | 400        | 400        | 400        |
| Folders                       | 200        | 47         | 21         |
| Max entries per folder        | 200        | 220        | 20         |
| Data (MB)                     | 129.0      | 129.0      | 129.0      |
| Allocated (MB)                | 130.9      | 130.3      | 130.2      |

Table 5: Datalake with the 200 real books (median of 5 runs).

The three layouts store exactly the same 129.0 MB in 400 files, with 0.9–1.5% of allocated overhead, and all three recover the 20 interrupted books with no losses or duplicates. Writing costs about the same in all of them (137–143 ms for 200 books); **book** is slightly slower because it creates 200 folders. The difference is in lookups: **book** and **range** compute the path from the id (28–30  $\mu$ s), while **time** has to walk its hour folders, newest first, and takes 378  $\mu$ s per book, 14 times more, a cost that grows with the hours of ingestion. **time**, in exchange, keeps every folder small (20 entries, against 200 in the root of **book** and 220 in the busiest range), which matters when ids are concentrated.

## 7.4 Metadata benchmark

Data: the generated rows of SPEC 10.2. **metadata\_insert** times inserting N rows in batches of 1,000, one transaction each, into an empty database; **metadata\_query** times 1,000 queries by id, by author and by title, picked with `JavaRandom(42)` among existing values. Each runs on **sqlite** (the SPEC schema) and **sqlite\_no\_index** (without the author and title indexes).

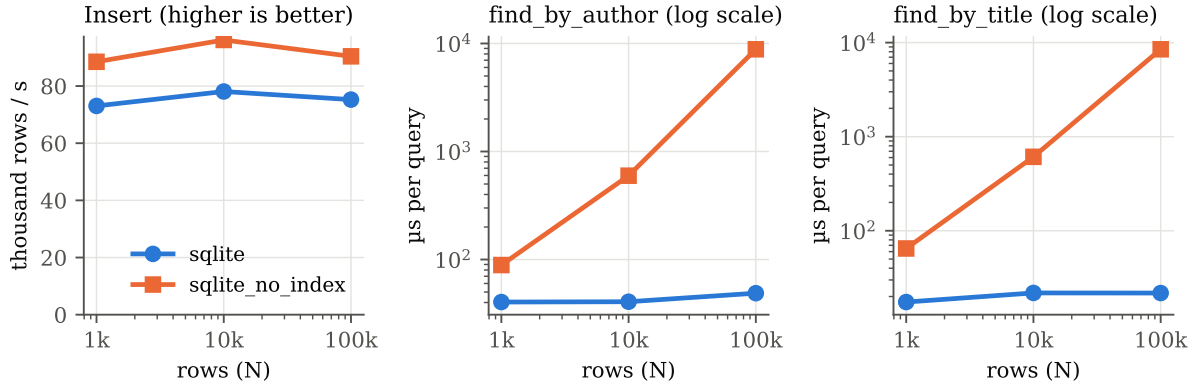


Figure 4: Metadata in SQLite with 1,000, 10,000 and 100,000 rows (median of 5 runs).

| Structure       | N       | Insert (rows/s) | by id ( $\mu$ s) | by author ( $\mu$ s) | by title ( $\mu$ s) |
|-----------------|---------|-----------------|------------------|----------------------|---------------------|
| sqlite          | 1,000   | 73,045          | 15.2             | 40.4                 | 17.5                |
| sqlite          | 10,000  | 78,095          | 13.4             | 40.7                 | 21.8                |
| sqlite          | 100,000 | 75,256          | 17.2             | 48.8                 | 21.8                |
| sqlite_no_index | 1,000   | 88,487          | 13.4             | 88.8                 | 64.8                |
| sqlite_no_index | 10,000  | 96,128          | 16.1             | 597.2                | 610.6               |
| sqlite_no_index | 100,000 | 90,399          | 19.7             | 8,834                | 8,490               |

Table 6: Metadata: insert throughput and average time per query (median of 5 runs).

With the SPEC indexes, every query stays flat as the table grows from 1,000 to 100,000 rows (13–17  $\mu$ s by id, 40–49 by author, 18–22 by title): SQLite walks a B-tree whatever N is. Without them, the queries by author and title become full scans and grow linearly with N, up to 8.8 ms per query with 100,000 rows, about 180 times slower. The price of the two indexes is on the insert side: 75,000 rows/s with them against 90,000 without (17% less). The indexes pay for themselves as soon as a table is queried more than it is written, which is the case of a search engine.

## 7.5 Inverted-index benchmark

Data: the prefixes of 50, 100 and 200 real books, tokenized once before any measurement, so the tokenizer is not in any timing.

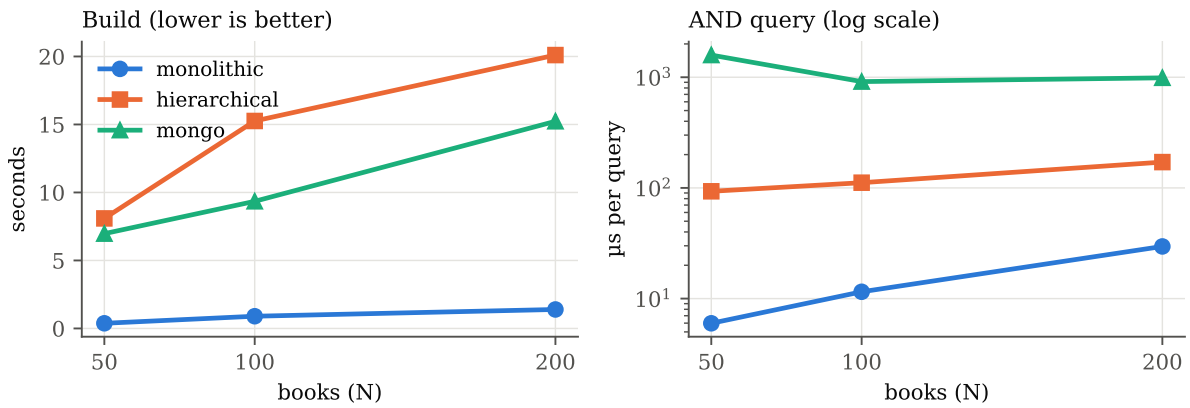


Figure 5: Building the index and answering the AND queries (median of 5 runs).

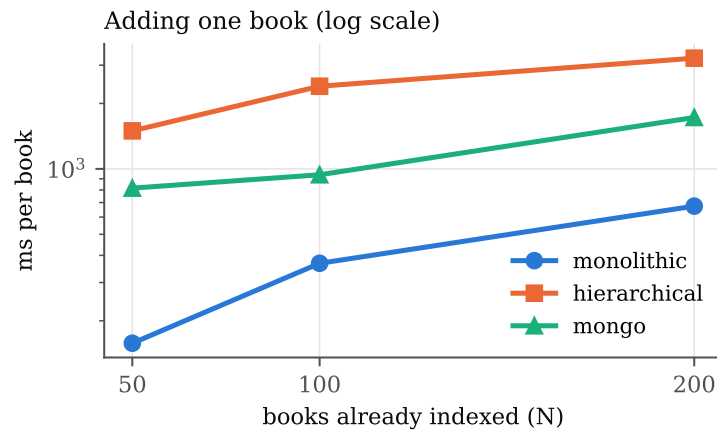


Figure 6: Cost of adding one book, with a flush per book, to an index that already holds 90% of N.

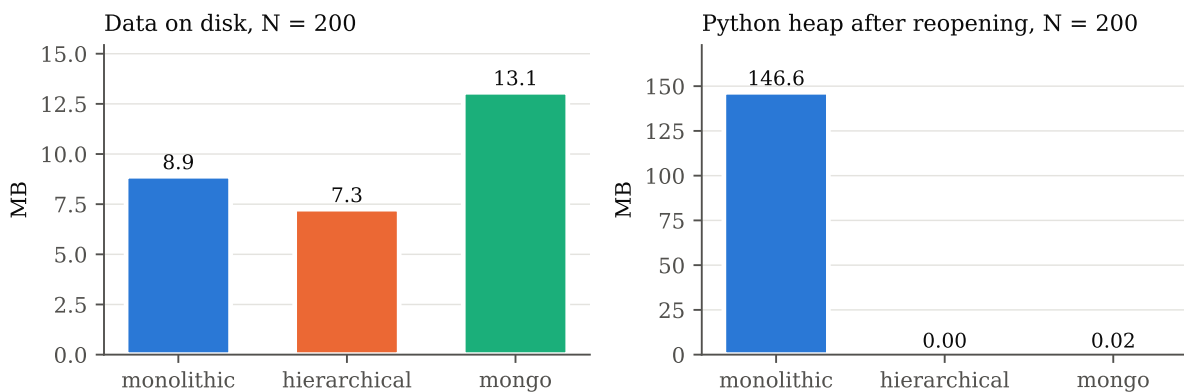


Figure 7: Disk usage and Python heap (`tracemalloc`) with N = 200.

| Structure    | N   | Build (ms) | Query ( $\mu$ s) | Update (ms/book) | Disk (MB) | Heap build (MB) | Heap open (MB) |
|--------------|-----|------------|------------------|------------------|-----------|-----------------|----------------|
| monolithic   | 50  | 379.3      | 6.0              | 157.7            | 1.90      | 32.6            | 35.5           |
| monolithic   | 100 | 898.6      | 11.5             | 367.8            | 3.75      | 61.4            | 65.2           |
| monolithic   | 200 | 1,395      | 29.6             | 674.0            | 8.89      | 117.3           | 146.6          |
| hierarchical | 50  | 8,091      | 93.2             | 1,496            | 1.15      | 0.0             | 0.0            |
| hierarchical | 100 | 15,256     | 111.5            | 2,400            | 2.74      | 0.0             | 0.0            |
| hierarchical | 200 | 20,103     | 171.1            | 3,230            | 7.25      | 0.0             | 0.0            |
| mongo        | 50  | 6,970      | 1,586            | 815.5            | 3.75      | 0.0             | 0.0            |
| mongo        | 100 | 9,349      | 913.3            | 939.5            | 6.39      | 0.0             | 0.0            |
| mongo        | 200 | 15,232     | 987.7            | 1,723            | 13.08     | 0.0             | 0.0            |

Table 7: Inverted index (median of 5 runs). Query: average per AND query; update: per added book; heap: `tracemalloc`, after building and after reopening.

**Build and query.** `monolithic` only fills dictionaries and writes one JSON, 1.4 s for 200 books; `hierarchical` creates 129,356 files (20.1 s) and `mongo` sends one upsert per term (15.2 s). Queries follow the same order: memory (30  $\mu$ s), one small file per term (171  $\mu$ s), one round trip to the server per term (988  $\mu$ s). **Update.** Adding one book with a flush costs 674 ms in `monolithic`, which rewrites the whole 8.9 MB file and grows with N (158 ms at N = 50); `hierarchical` rewrites one file per term of the book, about 7,900 on average (3.2 s); `mongo` 1.7 s. **Disk and memory.** The data take 7.3–13.1 MB in every structure, but the 129,356 small term files of `hierarchical` occupy 530 MB of 4 KB blocks, 73 times their size. `monolithic` holds 117–147 MB of Python objects for 200 books (a set of Python ints is far larger than the JSON), while `hierarchical` and `mongo` keep only the pending book and need almost nothing (<50 KB).

## 7.6 Comparison with Java and C++

The three implementations follow the same SPEC, measure the same books and write the same CSV format, so their results can be put side by side. Java and Python were measured on the same Linux machine; C++ was measured on macOS on Apple Silicon. A difference between Java and Python is therefore a difference of language and implementation, while a difference with C++ also includes the machine and the operating system.

A first attempt to produce the Python results on Windows 11 was many times slower than the Linux run: most of the time went into creating and deleting the  $\sim 129,000$  small files of the hierarchical index on NTFS, with Microsoft Defender scanning each one. For that reason the official Python results were taken on Linux, on the same machine as Java’s.

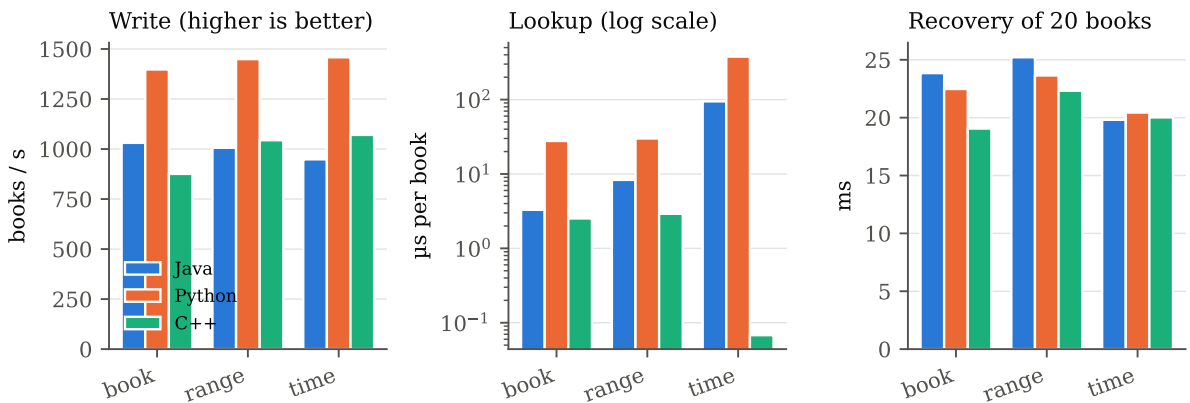


Figure 8: Datalake in the three languages, 200 real books (median of 5 runs).

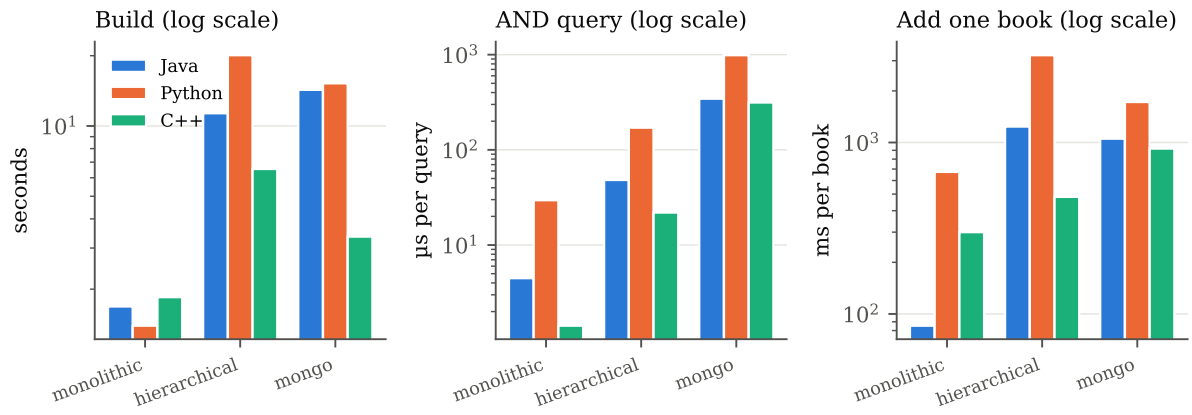


Figure 9: Inverted index in the three languages,  $N = 200$  (median of 5 runs, log scale).

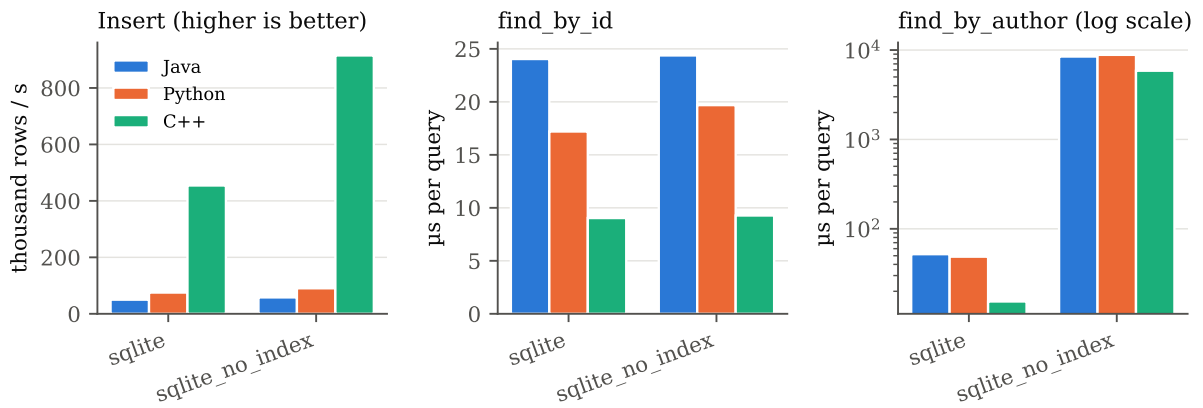


Figure 10: Metadata in the three languages, 100,000 generated rows (median of 5 runs).

| Median of 5 runs                                             | Java      | Python    | C++       |
|--------------------------------------------------------------|-----------|-----------|-----------|
| Datalake write, <b>book</b> (books/s)                        | 1,031     | 1,398     | 876       |
| Datalake lookup, <b>book</b> ( $\mu$ s/book)                 | 3.3       | 27.8      | 2.5       |
| Datalake lookup, <b>time</b> ( $\mu$ s/book)                 | 94.5      | 377.9     | 0.1       |
| Detect new books, <b>book</b> (ms)                           | 1.24      | 6.28      | 0.98      |
| Recovery, <b>book</b> (ms)                                   | 23.8      | 22.5      | 19.1      |
| Index build, <b>monolithic</b> , N = 200 (ms)                | 1,686     | 1,395     | 1,849     |
| Index build, <b>hierarchical</b> , N = 200 (ms)              | 11,339    | 20,103    | 6,546     |
| Index build, <b>mongo</b> , N = 200 (ms)                     | 14,286    | 15,232    | 3,359     |
| AND query, <b>monolithic</b> , N = 200 ( $\mu$ s)            | 4.5       | 29.6      | 1.4       |
| AND query, <b>hierarchical</b> , N = 200 ( $\mu$ s)          | 48.3      | 171.1     | 22.0      |
| AND query, <b>mongo</b> , N = 200 ( $\mu$ s)                 | 344.8     | 987.7     | 314.6     |
| Update, <b>monolithic</b> , N = 200 (ms/book)                | 85.4      | 674.0     | 300.0     |
| Update, <b>hierarchical</b> , N = 200 (ms/book)              | 1,239     | 3,230     | 482.0     |
| Update, <b>mongo</b> , N = 200 (ms/book)                     | 1,053     | 1,723     | 921.8     |
| Monolithic JSON, N = 200 (bytes)                             | 8,893,537 | 8,893,537 | 8,893,537 |
| Index terms, N = 200                                         | 129,356   | 129,356   | 129,356   |
| Metadata insert, <b>sqlite</b> , N = 100k (rows/s)           | 50,036    | 75,256    | 454,570   |
| find_by_author, <b>sqlite</b> , N = 100k ( $\mu$ s)          | 52.0      | 48.8      | 15.4      |
| find_by_author, <b>sqlite_no_index</b> , N = 100k ( $\mu$ s) | 8,460     | 8,834     | 5,869     |

Table 8: Key results in the three languages (median of 5 runs). The byte and term counts must be identical; the times are not expected to be.

**Same output.** The three languages report the same 129,356 terms, 1,581,064 postings and 8,893,537 JSON bytes, and the same storage counts: the differences below are of speed only.

**Java and Python (same Linux machine): differences of language.** Where the work happens in the operating system or in C code, Python is as fast as Java or faster: datalake writes (1,398 against 1,031 books/s), the monolithic build (1.4 against 1.7 s, where `json.dumps` runs in C), the Mongo build (15.2 against 14.3 s, bound by the server), SQLite inserts (75,000 against 50,000 rows/s, `executemany` in C against JDBC) and SQLite queries (49 against 52  $\mu$ s). Where an interpreted loop runs per call, Python is clearly slower: lookups (27.8 against 3.3  $\mu$ s, the cost of building `Path` objects and calling the file system from Python), AND queries (6.6 times slower on **monolithic**: sorting the set of each posting list and intersecting in Python), updates (674 against 85 ms per book: every flush sorts the ids of all 129,356 terms before writing) and the hierarchical build (1.8 times, a Python loop over every term file).

**C++ (macOS on Apple Silicon): differences of machine and design.** C++ is fastest in most experiments, but part of that gap comes from a different machine and operating system, so it cannot be read as a language difference alone. Some numbers also reflect design choices: its **time** lookup of 0.1  $\mu$ s comes from keeping a map of the books it has written instead of walking the hour folders, and its SQLite inserts (455,000 rows/s) depend on the SQLite build and the disk of that machine.

## 7.7 Summary: which structure to choose

| If the priority is...                                        | Datalake                                | Index                         |
|--------------------------------------------------------------|-----------------------------------------|-------------------------------|
| Looking books up by id                                       | <code>book</code> or <code>range</code> | —                             |
| Processing in arrival order, small folders                   | <code>time</code>                       | —                             |
| Fastest build, queries and updates at this scale (200 books) | —                                       | <code>monolithic</code>       |
| Index larger than memory, several processes querying         | —                                       | <code>mongo</code>            |
| Little disk space                                            | <code>any</code>                        | <code>monolithic</code>       |
| Avoiding many small files                                    | <code>range</code> or <code>time</code> | not <code>hierarchical</code> |

The structures chosen for the Python module are `book` and `monolithic`, the same as in Java and C++ (Section 3).

## 8. Conclusions and future improvements

### 8.1 Conclusions

The Python implementation delivers the complete Stage 1 data layer: crawler, datalake, metadata and inverted-index datamarts, AND search and a control layer that survives interruptions. One pipeline run fills the three datalake layouts and the three indexes, and its output matches the Java implementation byte for byte. The 12 benchmark experiments run with the methodology of the SPEC, check their own results, and are checked again by `verify_results`. The results agree with Java's: `book` and `monolithic` are the best choices at this scale, `mongo` is the way to grow, and `hierarchical` pays heavily for its thousands of small files. Compared with Java on the same machine, Python is competitive wherever the work is done by the operating system, SQLite, JSON or MongoDB, and several times slower in its own per-term loops, which is where a future version should spend its effort.

### 8.2 Limitations and future improvements

- **The cost of the interpreter in loops.** The tokenizer walks every character of every book in Python code, and the hierarchical index reads and writes files term by term. Both are interpreted loops, much slower than the same loops compiled in Java or C++. Moving the tokenizer to a compiled routine (or vectorising it over bytes) would keep the SPEC rules and remove most of that cost.
- **Tokenizing in parallel.** Books are independent, so tokenization could use `multiprocessing` (one process per core, avoiding the global interpreter lock) before the single-threaded indexing step.
- **Grouping writes.** The pipeline flushes the three indexes after every book, which is what makes the control layer safe, but rewrites the whole `monolithic` JSON 200 times. Flushing every K books and marking those K books together would keep the guarantee of SPEC section 8 with far fewer writes; for the hierarchical index, writing the term files of several books at once would reduce the number of file operations.
- **Memory limits of the `monolithic` index.** The whole index lives in a dictionary of Python sets, whose objects are much larger than the integers they hold. With thousands of books the index would no longer fit comfortably in memory, and the hierarchical or Mongo index (which keep only the pending book in memory) would become necessary.

- **Re-download on resume.** A book downloaded but not indexed when a run stops is downloaded again instead of being read from the datalake. Reading it back would match Java's "index what is downloaded first" and avoid a second copy in the **time** layout.
- **More unit tests.** The metadata repository, the monolithic and Mongo indexes and the search are covered by the benchmark checks and the pipeline output, but their unit tests only check the import; behavioural tests for them (Mongo ones skipped without a server) would catch regressions earlier.